ECOMMERCE POO

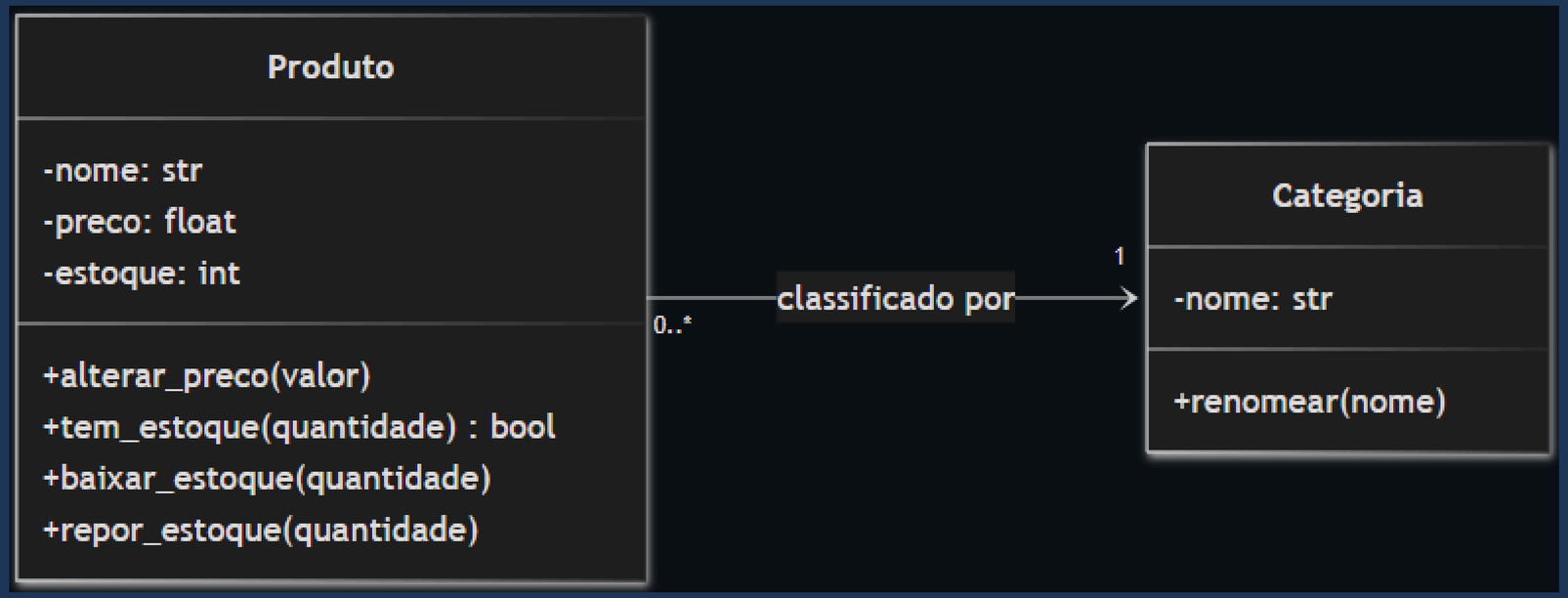
Análise e Desenvolvimento de Sistemas Web Mobile

Victor Gabriel

Diagramas UML(Linguagem de Modelagem Unificada)

Outros diagramas-
objeto
sequência
caso de uso

Visibilidade:
+ publico
- privado
protegido



Principais Notações e Símbolos
1 : Um e somente um (exatamente uma instância).
0..1 : Zero ou um (opcional).
* : Zero ou muitos (sem limite superior).
1..* : Um ou mais (pelo menos uma instância).
0..* : Zero ou mais (equivalente a usar apenas *).

0..* do lado de Produto: uma categoria pode estar relacionada a zero ou vários produtos.

1 do lado de Categoria: cada produto deve estar relacionado a exatamente uma categoria.

1. Classes
São modelos ou estruturas que defi_ nem as características e comportamentos de determinados objetos.

2. Atributos
São as informações ou características que uma classe possui.

3. Métodos
São as ações ou comportamentos que os objetos podem executar.

4. Objetos
São instâncias de uma classe, ou seja, elementos criados a partir de um modelo.

Exemplos no codigo:

```python
#Classe produto criada
class Produto:
```

```python
self.__nome = nome
self.__preco = 0.0
self.__categoria = categoria
self.__estoque = estoque
```

```python
def alterar_preco(self, valor):
    ...

def tem_estoque(self, quantidade):
    ...

def baixar_estoque(self, quantidade):
    ...

def repor_estoque(self, quantidade):
    ...
```

```python
produto1 = Produto("Notebook",
 3500, 10, "Eletrônicos")

produto2 = Produto("Mouse",
 80, 20, "Acessórios")
```

O que é Encapsulamento?

É o conceito de proteger os dados de um objeto,controlando como eles podem ser acessados e modificados.

1. Atributos privados:

```
self.__nome = nome
self.__preco = 0.0
self.__estoque = estoque
self.__categoria = categoria
```

2. Métodos que controlam as alterações:

```
def alterar_preco(self, valor):
    if valor <= 0:
        raise ValueError("Preço inválido")
    self.__preco = float(valor)
```

```
def baixar_estoque(self, quantidade):
    if not self.tem_estoque(quantidade):
        raise ValueError("Estoque insuficiente")
    self.__estoque -= quantidade
```

- Os atributos possuem __, indicando que são privados.
- O preço só pode ser alterado se for maior que zero.
- O estoque não pode ser reduzido além da quantidade disponível.

O que é Associação ?

É um relacionamento entre duas classes, permitindo que seus objetos interajam e trabalhem juntos

```python
def adicionar(self, produto, quantidade=1):

    item = self.__buscar_item(produto)

    if item is not None:
        item.aumentar(quantidade)
    else:
        self.__itens.append(ItemCarrinho(produto, quantidade))
```

O método adicionar() permite que um produto seja associado ao carrinho. Se o produto ainda não estiver nele, um novo item é criado. Caso já esteja, sua quantidade é aumentada.

```python
self.__carrinho = Carrinho()
```

No nosso projeto, o cliente possui um carrinho, e esse carrinho permite adicionar e remover produtos. Dessa forma, as classes trabalham juntas para realizar o processo de compra

O que é composição?

É uma relação em que uma classe é formada por objetos de outra classe, existindo uma dependência forte entre eles.

```python
def __init__(self):
    self.__itens = []    # lista de objetos ItemCarrinho
```

O carrinho possui uma estrutura interna para armazenar seus itens

```python
self.__itens.append(
    ItemCarrinho(produto, quantidade)
)
```

O objeto Carrinho cria um novo objeto ItemCarrinho e o armazena dentro da sua lista de itens

O que é herança?

A herança permite que uma classe herde atributos e métodos de outra classe, podendo reutilizá-los ou personalizá-los

- Pagamento é a classe mãe
- Pix é uma classe filha
- Cartao é outra classe filha

As duas classes herdam características de Pagamento, mas possuem suas próprias regras de cálculo

pix.py

```python
class Pix(Pagamento, Comprovante):
```

cartao.py

```python
class Cartao(Pagamento, Comprovante):
```

Classe abstrata

Uma classe abstrata funciona como um molde ou base estrutural para outras classes que possuem uma relação de herança direta com ela

3. Métodos abstratos

```python
@abstractmethod
def calcular_total(self):
```

```python
@abstractmethod
def descricao(self):
```

1. Declaração da classe abstrata

```python
from abc import ABC, abstractmethod

class Pagamento(ABC):
```

2. Atributo e validação

```python
def __init__(self, valor):
    if valor <= 0:
        raise ValueError("O valor precisa ser maior que zero.")
    self._valor = float(valor)
```

O que é Interface?

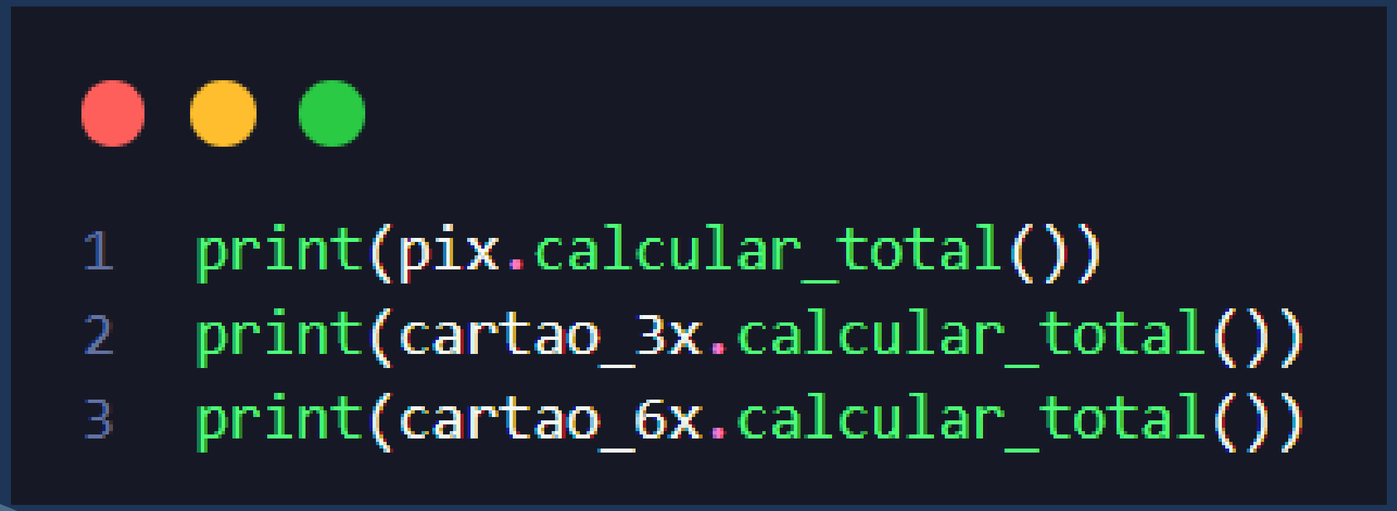
Uma Interface define um contrato que determina que todas as classes que herdarem dela precisam implementar um método

```python
from abc import ABC, abstractmethod
class Comprovante(ABC):
    @abstractmethod
    def gerar_comprovante(self):
```

- Define um contrato para geração de comprovantes.
- Utiliza uma classe abstrata para representar a interface.
- Exige que as classes filhas implementem gerar_comprovante().
- Permite que diferentes formas de pagamento gerem seus próprios comprovantes.

O que é polimorfismo?

O Polimorfismo permite que diferentes classes utilizem métodos com o mesmo nome, mas apresentem comportamentos diferentes de acordo com cada classe.



- Pix: aplica um desconto de 5%.
- Cartão: aplica uma taxa de 10% quando o pagamento é parcelado em mais de 3 vezes.

O que é agregação?

Agregação é um tipo de relacionamento entre classes em que um objeto possui ou utiliza outro objeto, mas ambos podem existir independentemente.

```python
class Pedido:
    def __init__(self, numero, cliente, forma_pagamento, parcelas=1):
        self.__numero = numero
        self.__cliente = cliente
```

- Pedido recebe um objeto Cliente pelo parâmetro cliente.
- O pedido guarda uma referência ao cliente em self.__cliente.
- O cliente não é criado dentro do pedido.
- O cliente pode continuar existindo independentemente do pedido.

Obrigado!